



中山大學

SUN YAT-SEN UNIVERSITY

数据安全和隐私保护

数据安全和隐私保护第二次实验

姓名 刘博文

学号 24337058

学院 网络空间安全学院

专业 网络空间安全

2026 年 9 月 10 日

目录

1	实验要求	1
2	实验原理	1
2.1	百万富翁问题	1
2.2	百万富翁问题原始求解思路	1
2.3	不经意传输 (Oblivious Transfer, OT)	2
3	实验设置	2
4	实验过程	2
4.1	总体设计: 可统计的模拟信道	3
4.2	密码学基础运算	3
4.3	任务一的实现	4
4.4	任务二: n 选 1 OT 的实现	6
4.5	任务二: 用 OT 求解百万富翁问题	7
4.6	协议流程演示	8
4.7	正确性测试	8
4.8	性能对比	9
4.9	可扩展性分析	11
4.10	半诚实模型下双方的可见信息	12
4.11	为何不应泄露未选消息、选择位与额外财富信息	13
4.12	隐私性实证结果	13
5	实验总结	14

1 实验要求

1、使用 Python 或其他熟悉的语言完成实验，可调用密码库中的基础运算。两个任务均输出“Alice 更富有”“Bob 更富有”或“双方财富相等”，不得输出对方财富及差值。

2、任务一为解决百万富翁问题。实现基于公钥变换、随机数和交互判断的原始求解方案，说明密钥生成、消息构造、比较判定及相等情况的处理。不得以明文交换、共享变量直接比较或可信第三方代算替代协议。

3、任务二则是用 OT 解决百万富翁问题。先实现 n 选 1 OT，可采用 RSA 或 Diffie-Hellman 体系实现，需说明采用的具体协议与参数。随后使用 OT 处理百万富翁问题。

4、比较实验结果并分析。在相同设备、输入和可比密码参数下，记录两种方案的正确率、耗时、通信字节数及交互轮数。并对两种方案的隐私性进行分析，列出双方可见信息，解释为何不应泄露未选消息、选择位或额外的财富信息。

5、实验要求提交完整代码和实验报告的 PDF 文件，实验报告需包括实验原理、实验环境、两项任务的实现方式、关键代码解释、测试结果比较和分析，提交方式以后续课程通知为准，截止时间 10.8 晚 24 时。

2 实验原理

2.1 百万富翁问题

百万富翁问题是 Andrew Yao 在 1982 年提出的，一般被认为是安全多方计算 (Secure Multi-party Computation, MPC) 的起点。问题本身很直观：Alice 和 Bob 各有一个财富值 a 和 b ，两人想知道谁更富，但谁都不愿意把自己的具体数字告诉对方。

要算的函数就是比较： $a > b$ 输出 Alice 更富有， $a < b$ 输出 Bob 更富有， $a = b$ 输出双方财富相等。结果本身以及由结果必然能推出来的东西可以知道，除此之外不应该多知道任何信息。我们这门课和这次实验都按半诚实模型来做，也就是双方会老老实实按协议走，但私下会拿收到的消息反复琢磨。

2.2 百万富翁问题原始求解思路

原始方案的思路是用公钥变换把 Bob 的财富索引藏起来：Bob 用自己的随机数把财富盲化后发给 Alice，Alice 对一串候选位置做私钥运算，再用一个素数把结果压缩，并按自己的财富对其中一部分结果做调整；Bob 最后只在自己那一位上做检验，就能判断出大小。实现的时候有几处容易踩坑：模运算的边界、余数之间的间距检验、以及不满足条件时的失败重试。另外，单跑一次只能区分 $a \geq b$ 和 $a < b$ ；要区分相等，得交换角色、换一批全新随机数再跑一次。

2.3 不经意传输 (Oblivious Transfer, OT)

不经意传输是 Michael Rabin 在 1981 年提出的，算是一种很基础的密码学协议，核心想法是“选择性隐私传输”：

发送方手里有两条消息 m_0, m_1 。接收方选一个索引 b ，只拿到 m_b ；

同时发送方不知道 b 是几，接收方也拿不到另一条 m_{1-b} ；

在 1-out-of-2 OT 里，通常用公钥加密或同态加密来实现。

一个简单的公钥实现大概长这样：

1. 接收方先生成一对公私钥，把加密过的选择信息发给发送方。
2. 发送方拿这个信息算出两个加密消息，使得只有一个能被正确解密。
3. 接收方用自己的私钥解出选中的那条，另一条还是加密状态。

3 实验设置

- 操作系统：Windows-11-10.0.26100-SP0
- 处理器：Intel64 Family 6 Model 183 Stepping 1, GenuineIntel
- 编程语言：Python 3.13.14（仅使用标准库）
- 密码学基础运算：Miller-Rabin 素性检测、随机大素数生成、RSA 密钥对生成、模幂运算都是自己写的（见代码 2），没有调用第三方密码库
- 散列函数：hashlib.sha256，用于 OT 掩码密钥流（MGF1 风格）
- 随机数：secrets.SystemRandom（操作系统密码学安全随机源）
- 密钥参数：RSA 模数 1024 / 2048 位，公钥指数 $e = 65537$ ；Yao 方案中压缩素数 p 取 $|n|/4$ 位
- 排版工具：XeLaTeX（MiKTeX 发行版）+ SYSU 实验报告模板，插图用 TikZ 画成矢量图

4 实验过程

工程结构是这样分的：`common.py`（公共基础：信道、数论、RSA、掩码）、`yao_millionaires.py`（任务一）、`ot_rsa.py`（ n 选 1 OT）、`ot_millionaires.py`（用 OT 解百万富翁问题）、`run_experiments.py`（实验入口）。

4.1 总体设计：可统计的模拟信道

为了能客观比较通信开销和交互轮数，我没有让两方直接共享变量，而是让所有消息都必须过一个可统计的信道。信道按“序列化后的字节数”累加通信量，每发一条消息记一轮，同时留一份可审计的消息日志（表 1、表 2）。

```
1 class Channel:
2     # Simulated channel: meters traffic (bytes) and interaction
      rounds.
3     def __init__(self):
4         self.bytes = 0          # total transmitted bytes
5         self.rounds = 0        # total number of messages
6         self.log = []          # auditable message log
7
8     def send(self, sender, receiver, payload, note=""):
9         size = wire_size(payload)      # serialized length in
      bytes
10        self.bytes += size
11        self.rounds += 1
12        self.log.append({"seq": self.rounds, "from": sender,
13                          "to": receiver, "note": note, "bytes": size
14                          })
15
16        return payload
17
18 def wire_size(obj):
19     # Wire size of int / bytes / str / list / tuple / dict.
20     if isinstance(obj, int):
21         return max(1, (obj.bit_length() + 7) // 8)
22     if isinstance(obj, (bytes, bytearray)):
23         return len(obj)
24     if isinstance(obj, str):
25         return len(obj.encode("utf-8"))
26     if isinstance(obj, (list, tuple, set)):
27         return sum(wire_size(x) for x in obj)
28     ...
```

Listing 1: 可统计的模拟信道（common.py 节选）

4.2 密码学基础运算

```

1 # Miller-Rabin probabilistic primality test
2 def is_probable_prime(n, rounds=40):
3     for p in _SMALL_PRIMES:
4         if n == p: return True
5         if n % p == 0: return False
6     d, s = n - 1, 0
7     while d % 2 == 0:
8         d //= 2; s += 1
9     for _ in range(rounds):
10        a = _rng.randrange(2, n - 1)
11        x = modexp(a, d, n)
12        if x == 1 or x == n - 1: continue
13        for _ in range(s - 1):
14            x = x * x % n
15            if x == n - 1: break
16        else:
17            return False
18    return True
19
20
21 # Generate an RSA key pair of `bits` bits
22 def gen_rsa_keypair(bits, e=65537):
23     while True:
24         p = gen_prime(bits // 2)
25         q = gen_prime(bits // 2)
26         if p == q: continue
27         n = p * q
28         phi = (p - 1) * (q - 1)
29         if math.gcd(e, phi) != 1: continue
30         return RSAKey(n, e, pow(e, -1, phi), bits)

```

Listing 2: Miller-Rabin 素性检测与 RSA 密钥生成

4.3 任务一的实现

我把加密方和判定方拆成了两个独立的类 YaoEncryptor / YaoDecider, 各自只持有自己的私有输入, 任何一方在代码层面就拿不到对方的财富变量。

```

1 # --- Bob (decider): blind his own wealth with a fresh random x ---
2 def step2_blind(self, ch, peer, n, e):

```

```

3     x = rand_below(n)                    # fresh randomness, |x| ~ |n|
4     C = modexp(x, e, n)                  # public-key transform C = x^e
        mod n
5     k = (C - self.wealth) % n            # modulo boundary: C - b may
        be negative
6     ch.send(self.name, peer, k, "k=x^e mod n - b")
7     return k

```

Listing 3: 任务一步骤 2: 判定方盲化自己的财富

代码 3 里, x 每一轮都是全新随机的; $k = (C - b) \bmod n$ 显式取了模, 就是为了处理 $C - b$ 可能是负数这种模运算边界。

```

1 # --- Alice (encryptor): private-key op, mod-p folding, then perturb
   ---
2 def step3_transform(self, ch, peer, k, N, p_bits):
3     n, d = self.key.n, self.key.d
4     nums = [(k + i) % n for i in range(1, N + 1)] # modulo boundary
5     while True:                                   # retry with a
6         fresh prime
7         p = gen_prime(p_bits)
8         Y = [modexp(v, d, n) for v in nums]         # N private-key
9         modexps
10        Z = [y % p for y in Y]
11        ok = True
12        if Z and max(Z) + 1 >= p:                   # (b) no wrap
13            around p
14            ok = False
15        if ok:                                       # (a) pairwise
16            gap >= 2
17            srt = sorted(Z)
18            for i in range(1, len(srt)):
19                if srt[i] - srt[i - 1] < 2:
20                    ok = False
21                    break
22        if ok:
23            break
24        # perturb: keep Z_i for i <= a, add 1 for i > a
25        W = [Z[i] if (i + 1) <= self.wealth else Z[i] + 1 for i in range(
26            N)]
27        ch.send(self.name, peer, [p] + W, "prime_p_and_N_folded_values")

```

23

```
return p, W
```

Listing 4: 任务一步骤 3: 私钥运算、模素数压缩、间距检验与扰动

代码 4 有三个地方我觉得比较关键:

1. $(k + i) \bmod n$ 统一取模, 免得 $k + i$ 越过模数边界;
2. 间距检验加失败重试: 要求任意两个 Z_i 相差至少 2、且 $Z_i + 1 < p$, 这样“加 1”既不会和其他 Z_j 撞上, 也不会越过模数边界; 不满足就换个新素数 p 重算。取 $|p| = |n|/4$ 的时候这个条件基本一次就过 (实测每轮就 1 次尝试)。
3. 按自己的财富做扰动: $i \leq a$ 原样发, $i > a$ 加 1, 这是整个协议里唯一泄露大小关系的地方, 而且只泄露 1 比特。

```

1 # --- Bob (decider): only inspect his OWN slot ---
2 def step4_decide(self, ch, peer, p, W):
3     G = self._x % p                                # Y_b = (k+b)^d = (x^e)^d =
4     x                                              # True  <=>  w_p >= w_q
5     result = (W[self.wealth - 1] == G)
6     ch.send(self.name, peer, 1 if result else 0, "1-bit_decision")
7     return result

```

Listing 5: 任务一步骤 4: 判定方只检查自己的位置

完整求解时交换角色再跑一轮, 两轮结果合起来就能区分三态 (见 `yao_millionaires`)。

4.4 任务二: n 选 1 OT 的实现

```

1 # ---- Round 1: sender -> receiver
2 -----
3 def round1(self, ch, peer):
4     N, e = self.key.n, self.key.e
5     self.xs = [rand_below(N) for _ in range(self.n_msg)] # n
6     blinding factors
7     ch.send(self.name, peer, (N, e, self.xs), "public_key_and_x_i")
8     return N, e, self.xs
9
10 # ---- Round 2: receiver -> sender
11 -----
12 def round2(self, ch, peer, N, e, xs):
13     k = rand_below(N)
14     self.k = k

```

```

12     y = xs[self.sigma] * modexp(k, e, N) % N      # y = x_sigma * k^e
           mod N
13     ch.send(self.name, peer, y, "blinded_choice")
14     return y
15
16 # ---- Round 3: sender -> receiver
    -----
17 def round3(self, ch, peer, y):
18     N, d = self.key.n, self.key.d
19     ciphers = []
20     for i in range(self.n_msg):
21         k_i = modexp(y * pow(self.xs[i], -1, N) % N, d, N) # (y/x_i)
                ^d
22         ciphers.append(mask(self.messages[i], k_i))        # m_i
                XOR H(k_i)
23     ch.send(self.name, peer, ciphers, "n_encrypted_messages")
24     return ciphers
25
26 # ---- Local decryption: only index sigma works
    -----
27 def decrypt(self, ciphers):
28     return unmask(ciphers[self.sigma], self.k)

```

Listing 6: 基于 RSA 的 1-out-of-n OT 三轮实现 (ot_rsa.py 节选)

实现上有几个点:

- 盲化因子 x_i 和密钥一样是一次性的, 每次协议都重新生成;
- 模逆直接用 $\text{pow}(a, -1, N)$; $k_i = (y \cdot x_i^{-1})^d \bmod N$ 一共 n 次私钥模幂, 这是 OT 计算开销的大头;
- 消息统一填成定长 32 字节块再异或掩码, 把长度侧信道抹掉;
- 掩码是 $\text{SHA256}(\text{domain} \parallel k_i \parallel \text{ctr})$ 级联起来的 MGF1 风格密钥流。

4.5 任务二: 用 OT 求解百万富翁问题

```

1 # MSG_ALICE_RICHER / MSG_EQUAL / MSG_BOB_RICHER are the three
2 # allowed outputs of the millionaires problem.
3 def compare_label(a, i):
4     if i < a: return MSG_ALICE_RICHER    # i < a -> Alice is richer

```

```

5     if i == a: return MSG_EQUAL          # i == a -> equal
6     return MSG_BOB_RICHER              # i > a -> Bob is richer
7
8 def build_ot_messages(a, N):
9     # m_i = cmp(a, i) for i = 1..N, padded to a fixed 32-byte block
10    return [pad_block(compare_label(a, i), MESSAGE_BLOCK)
11            for i in range(1, N + 1)]
12
13 def ot_millionaires(a, b, N, bits=1024):
14     ch = Channel()
15     alice = OTSender("Alice", build_ot_messages(a, N), bits=bits)
16     bob = OTReceiver("Bob", b - 1)      # sigma = b - 1
17     N_mod, e, xs = alice.round1(ch, bob.name)
18     y = bob.round2(ch, alice.name, N_mod, e, xs)
19     ciphers = alice.round3(ch, bob.name, y)
20     result = unpad_block(bob.decrypt(ciphers)) # = cmp(a, b)
21     ch.send("Bob", "Alice", pad_block(result, MESSAGE_BLOCK), "result")
22     return result, ch

```

Listing 7: 用 1-out-of-N OT 求解百万富翁问题 (ot_millionaires.py 节选)

代码 7 里, Alice 只要把 N 条消息准备好, Bob 用 $\sigma = b - 1$ 取一次, 拿到的就是最终三态结果。最后 Bob 把这个结果 (本来就是允许公开的输出) 定长回传给 Alice, 两边结论就一致了。

4.6 协议流程演示

取 $N = 10$ 、 $a = 7$ 、 $b = 3$ 、RSA 1024 位, 两个方案都输出 “Alice 更富有”, 和明文参考结果对得上。消息交互日志如下。

从表里能看出来: Yao 方案两轮一共 8 条消息, 其中第 3、7 条是 N 个压缩值, 占了通信量的大头; OT 方案只有 4 条消息, 但第 1 条就要发 N 个盲化因子 (每个 $|N|$ 位), 通信量反而更大。这正好是两种方案最核心的权衡: Yao 通信量小, 但交互轮数多、计算量大; OT 轮数少、计算量小, 但通信量大。

4.7 正确性测试

RSA 1024 位下, $N = 10$ 时把 100 组 (a, b) 全穷举了一遍、 $N = 100$ 时随机抽了 106 组 (含边界用例), 两个方案的正确率都是 100.00%; 1-out-of-8 OT 把所有选择都遍历了一遍, 重复 20 轮共 160 次, 也全对。三种输出 (Alice 更富有 / Bob 更富有 / 双方财富相等) 的分布和正确数见表 4, 说明相等的情况也处理对了。

序号	发送方	接收方	消息含义	字节数
1	Alice	Bob	步骤 1 RSA 公钥 (n, e)	131
2	Bob	Alice	步骤 2 $k = x^e \bmod n - b$ (财富被随机数盲化)	128
3	Alice	Bob	步骤 3 素数 p 与 N=10 个压缩值 W_i (已按自身财富扰动)	352
4	Bob	Alice	步骤 4 判定结果 1 比特 (True = 加密方财富 $\geq$ 判定方财富)	1
5	Bob	Alice	步骤 1 RSA 公钥 (n, e)	131
6	Alice	Bob	步骤 2 $k = x^e \bmod n - b$ (财富被随机数盲化)	128
7	Bob	Alice	步骤 3 素数 p 与 N=10 个压缩值 W_i (已按自身财富扰动)	352
8	Alice	Bob	步骤 4 判定结果 1 比特 (True = 加密方财富 $\geq$ 判定方财富)	1

表 1: 任务一 Yao 方案的消息交互日志 ($N = 10$, $a = 7$, $b = 3$, RSA 1024 位; 共 8 轮 / 1,224 B)

序号	发送方	接收方	消息含义	字节数
1	Alice	Bob	OT 第 1 轮公钥 (N,e) 与 n=10 个盲化因子 x_i	1,411
2	Bob	Alice	OT 第 2 轮 $y = x_{\sigma} \cdot k^e \bmod N$ (选择被随机 e 次幂盲化)	128
3	Alice	Bob	OT 第 3 轮 n=10 条加密消息 $c_i = m_i \oplus H(k_i)$	320
4	Bob	Alice	结果回传双方共同输出 (三态结果本身)	32

表 2: 任务二 OT 方案的消息交互日志 ($N = 10$, $a = 7$, $b = 3$, RSA 1024 位; 共 4 轮 / 1,891 B)

测试项	规模	用例数	正确数	正确率
穷举全部 (a, b) (两方案合计)	$N = 10$	200	200	100.00%
任务一 Yao 随机 + 边界用例	$N = 100$	106	106	100.00%
任务二 OT 随机 + 边界用例	$N = 100$	106	106	100.00%
1-out-of-8 OT 遍历全部选择	$n = 8$	160	160	100.00%

表 3: 正确性测试结果 (RSA 1024 位)

4.8 性能对比

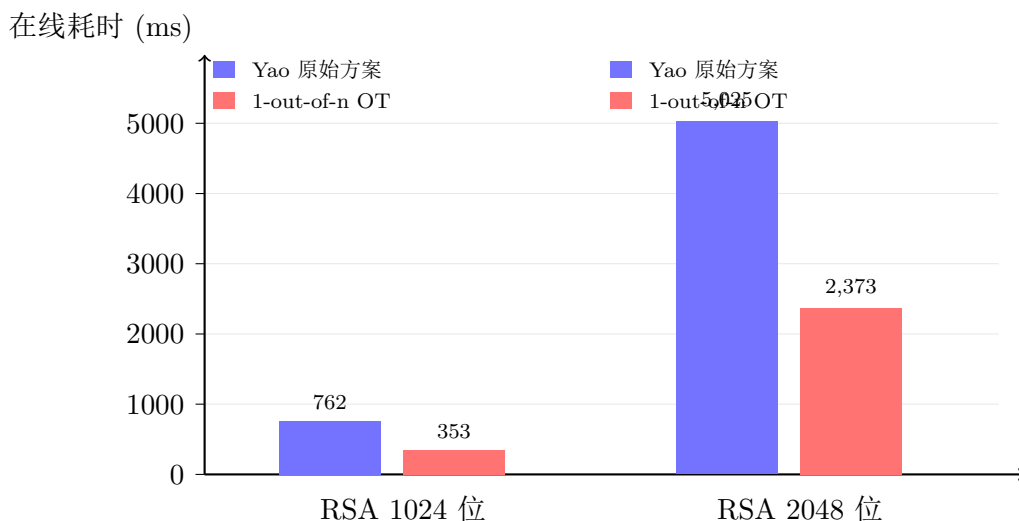
在同一台机器、相同输入 ($N = 100$ 、 $a = 73$ 、 $b = 28$)、相同密码参数下对比两种方案，结果见表 5 和图 1。

- 耗时: OT 的在线耗时大约是 Yao 的 46%~47%。原因是 Yao 要跑两轮 (每轮 N 次私钥模幂，加起来约 $2N$ 次)，而 OT 只要一轮 (N 次私钥模幂)。实测模幂次数: Yao 约 308 次，OT 约 101 次，和理论上 $2N$ 对 N 的关系对得上。

比较结果	用例数	Yao 正确数	OT 正确数
Bob 更富有	43	43	43
双方财富相等	18	18	18
Alice 更富有	45	45	45

表 4: $N = 100$ 随机测试中三种输出的分布与正确数

方案	RSA 位数	在线耗时 (ms)	含密钥生成 (ms)	通信量 (B)	交互轮数	模型
任务一 Yao 原始方案	1024	762.38	918.40	6,984	8	模型
任务二 1-out-of-n OT	1024	352.65	453.20	16,290	4	模型
任务一 Yao 原始方案	2048	5,024.91	6,666.58	13,959	8	模型
任务二 1-out-of-n OT	2048	2,372.85	3,231.32	29,346	4	模型

表 5: 两种方案的性能与通信开销对比 ($N = 100$, $a = 73$, $b = 28$)图 1: 两种方案的在线耗时对比 ($N = 100$, 不含一次性密钥生成开销)

- **通信量:** Yao 约 6,984 B (1024 位), OT 约 16,290 B, OT 差不多是 Yao 的 2.3 倍。因为 OT 第 1 轮要发 N 个 $|N|$ 位的盲化因子 x_i , 而 Yao 传的是 N 个 $|p| = |n|/4$ 位的压缩值。
- **交互轮数:** Yao 8 轮, OT 4 轮 (含结果回传), 直接少一半。网络延迟大的时候, 这一点往往比字节数更值钱。
- **密钥生成:** Yao 要 2 把 RSA 密钥 (双方各当一次加密方), OT 只要 1 把。RSA-1024 单次密钥生成约 93.8 ms, RSA-2048 约 721.3 ms, 属于一次性开销, 密钥复用一下就能摊薄。

- **安全参数的影响：**从 1024 位到 2048 位，两者耗时都涨了约 6.6~6.7 倍，和模幂运算 $O(|n|^2) \sim O(|n|^3)$ 的复杂度增长对得上，相对优劣关系没变。

4.9 可扩展性分析

财富上界 N 直接决定要处理多少个候选位置。图 2 和图 3 是 $N = 10, 25, 50, 100$ 时两种方案的耗时和通信量，原始数据在表 6。

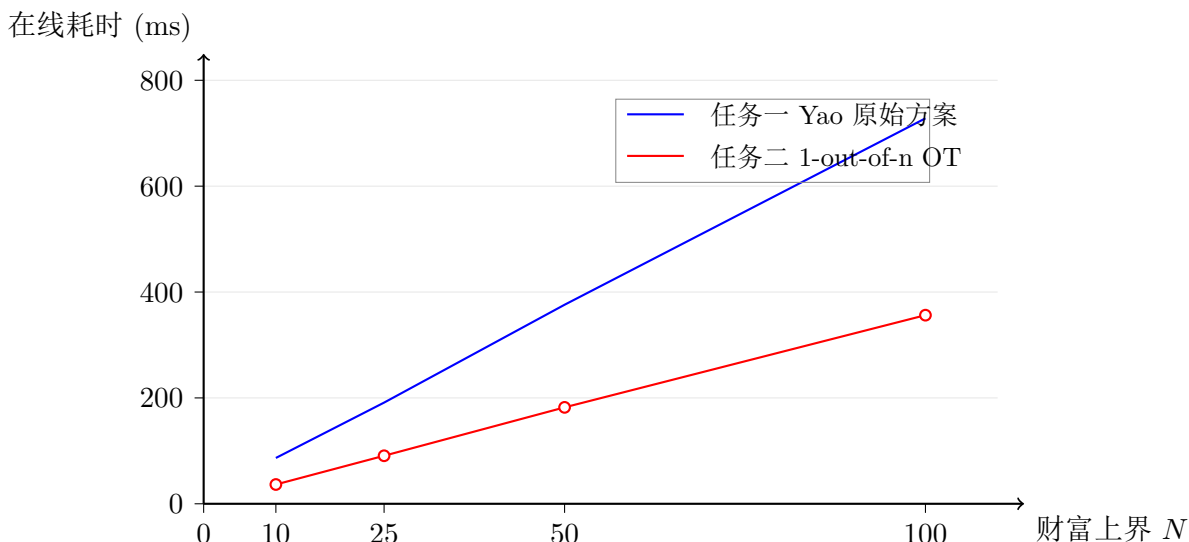


图 2: 财富上界 N 对在线耗时的影响 (RSA 1024 位)

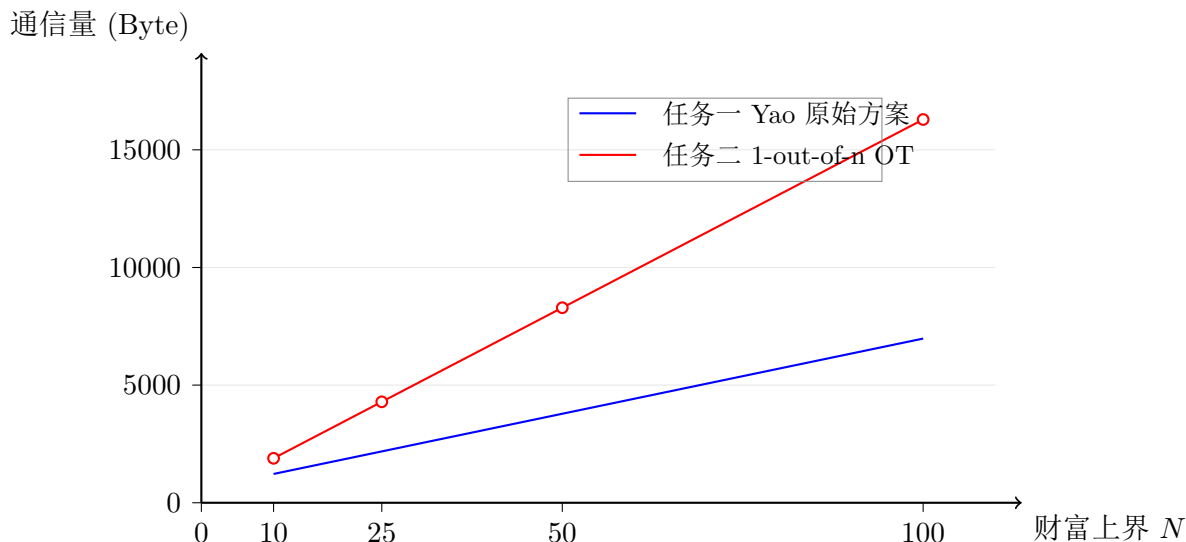


图 3: 财富上界 N 对通信量的影响 (RSA 1024 位)

两者的耗时和通信量都随 N 线性增长，但斜率不一样：耗时的斜率比大概是 2 : 1 (Yao 跑两轮)，通信量的斜率比大概是 1 : 2.3 (Yao 传 $|p| = \frac{1}{4}|n|$ 位的压缩值，OT 传 $|N|$ 位的盲化因子)。也就是说，财富范围很大的时候，Yao 在通信量上占优，OT 在耗

财富上界 N	Yao 耗时 (ms)	Yao 通信量 (B)	Yao 轮数	OT 耗时 (ms)	OT 通信量 (B)	OT 轮数
10	86.51	1,224	8	36.44	1,890	4
25	191.15	2,184	8	90.75	4,291	4
50	376.01	3,783	8	182.09	8,291	4
100	728.16	6,979	8	356.35	16,290	4

表 6: 财富上界 N 对耗时与通信量的影响 (RSA 1024 位)

时和轮数上占优。实际工程里可以用“分桶 / 按位比较 + OT 扩展”之类的办法把 OT 的通信量降到 $O(\log N)$ 。

4.10 半诚实模型下双方的可见信息

方案	参与方	可见信息
任务一 Yao	Alice	自己的财富 a 两轮各一个盲化值 $k = x^e - b \bmod n$ 自己生成的 RSA 私钥 d 对方回传的 1 比特判定结果 最终三态输出
	Bob	自己的财富 b Alice 的 RSA 公钥 (n,e) 素数 p 与 N 个压缩值 W_i (其中只有自己那一位可解读) 最终三态输出
任务二 OT	Alice	自己的财富 a 自己的 RSA 密钥对 盲化选择 $y = x_\sigma \cdot k^e \bmod N$ 最终三态输出
	Bob	自己的财富 b 公钥 (N,e) 与 N 个盲化因子 x_i N 条密文 c_i (只能解开第 b 条) 最终三态输出

表 7: 半诚实模型下双方的可见信息清单

4.11 为何不应泄露未选消息、选择位与额外财富信息

1. **未选消息（OT 的发送方隐私）**：在百万富翁场景里，Alice 的 N 条消息 $m_i = \text{cmp}(a, i)$ 其实把她的财富完整编码进去了——只要能读出足够多条，二分一下就能定出 a 的精确值。所以接收方一旦能解开没选的那些消息，协议就退化成“明文交换”了，隐私目标直接落空。OT 靠“每条消息用不同的 k_i 掩码，而且只有 k_σ 能解”来堵这条通路。
2. **选择位 σ （OT 的接收方隐私）**： $\sigma = b - 1$ 就是 Bob 的财富本身。发送方要是能认出 σ ，Bob 的私有输入就全暴露了。协议里 $y = x_\sigma \cdot k^e \bmod N$ 被随机的 e 次幂盲化，在 RSA 假设下和 σ 独立，所以选择位是保住的。
3. **额外的财富信息（两种方案共有）**：协议允许泄露的只有比较结果本身以及它的必然推论（比如由“Alice 更富有”能推出 $a > b$ ，由“相等”能推出 $a = b$ ）。除此之外，关于 a 或 b 的具体取值、差值、落在哪个区间之类的信息都不该漏。
 - Yao 方案里，Alice 收到的是被随机 x 盲化过的 k 。她虽然有私钥 d ，但不知道 x ，没法判断 N 个 Y_i 里哪个是 x ，所以学不到 b ；Bob 只掌握自己那一位的对应关系，其余 W_i 因为没 d 也解不出来，所以学不到 a 。
 - OT 方案里，安全性直接归约到 OT 原语的双向保护，而且消息定长填充，长度侧信道也堵住了。
4. **相等判定的代价**：Yao 方案为了区分相等，必须交换角色再跑一轮，这样双方会各多知道一个 1 比特的中间结论（ $a \geq b$ 和 $b \geq a$ ）。但这两个其实都被最终输出蕴含了，不算额外泄露。OT 方案一轮就能拿到三态结果，暴露面更小。

4.12 隐私性实证结果

隐私目标	实验方法	实测结果
发送方不可知选择 σ	对 $\sigma = 0, 1$ 各采样 300 个 y ，训练经验最优阈值分类器	两类样本均值 0.4
接收方不可知未选消息	用自己掌握的 k 逐条解密全部密文	可识别明文 1 条

表 8: 隐私性实证结果（RSA 1024 位）

接收方拿自己的 k 试着把全部密文都解了一遍，结果如下：

- c_0 解密前 24 hex: 06b8c33c4192358603213edf（乱码，不可读）
- c_1 解密前 24 hex: e58f8ce696b9e8b4a2e5af8c（选中，可正确解密）
- c_2 解密前 24 hex: 844930d00a46871b6bb7e3b1（乱码，不可读）

这里要补一句：上面那个“分类器准确率”只是一次具体的攻击尝试，用来直观说明 y 对 σ 没有可用的统计偏差；发送方隐私的严格保证还是来自 RSA 假设下的不可区分性，而不是某一次统计检验。

5 实验总结

这次实验把百万富翁问题的两种解法都做了一遍，也做了比较系统的对比：

1. **任务一**实现的是 Yao 原始方案：用 RSA 公钥变换把财富索引藏起来，中间经过私钥运算、模素数压缩、间距检验与失败重试、按自身财富加 1 扰动，最后判定方只查自己那一位就能得出大小关系；相等的情况则通过交换角色、换全新随机数和密钥再跑一轮来处理。
2. **任务二**实现了基于 RSA 的 1-out-of- n OT (EGL 构造)，并给出一种只要 1 次 OT、3 轮交互就能直接输出三态结果的百万富翁解法。
3. **正确性**：两方案在穷举和小样本随机测试里正确率都是 100%，相等情况也都处理对了。
4. **性能**：OT 耗时差不多是 Yao 的一半，交互轮数也减半；但通信量约是 Yao 的 2.3 倍（1024 位下 16,290 B 对 6,984 B）。两者随财富上界 N 都是线性增长。
5. **隐私性**：半诚实模型下两方案都只泄露比较结果及其必然推论。OT 方案的安全性可以很清晰地归约到原语的双向保护，结构更干净、也更好做形式化论证；Yao 原始方案则依赖“随机数 x 未知”和“私钥 d 不在手里”这两点，论证相对绕一些，而且还要多跑一轮来处理相等。

做完这次实验，我对安全两方计算的基本想法有了更具体的感受：“不泄露输入而只算出函数”是真的能做到的，代价就体现在计算、通信和交互轮数上；而 OT 作为基础原语，能把比较复杂的安全计算问题规整地归约到少数几个密码学假设上。

参考文献

- [1] A. C. Yao, “Protocols for Secure Computations,” in *Proc. 23rd Annual Symposium on Foundations of Computer Science (FOCS)*, 1982, pp. 160–164.
- [2] M. O. Rabin, “How to Exchange Secrets with Oblivious Transfer,” Technical Report TR-81, Aiken Computation Laboratory, Harvard University, 1981.
- [3] S. Even, O. Goldreich, and A. Lempel, “A Randomized Protocol for Signing Contracts,” *Communications of the ACM*, vol. 28, no. 6, pp. 637–647, 1985.